Abstract

Managing personal finances is often challenging due to inconsistent spending habits and the limitations of traditional financial tracking tools. To address this, we propose a Multi-Agent Finance Manager, a modular and intelligent system designed to help users efficiently manage their income, expenses, savings, investments, and long-term financial goals. The system employs a multi-agent architecture, where generic financial agents evaluate user data using rule-based logic, while specialized agents leverage domain knowledge to generate deeper financial insights. This layered approach enables more accurate and context-aware financial analysis. A key feature of the platform is category-based goal management, allowing users to define, track, and achieve financial objectives across multiple domains such as savings, investments, and emergency funds. A milestone-based reward mechanism is integrated to promote disciplined financial behavior and long-term consistency. To enhance decision-making, the system integrates real-time financial news through a News API, enabling users to stay informed about current economic developments. A dedicated Market Analysis Agent processes this information and aligns it with the user's financial profile to deliver personalized, context-aware recommendations. In addition, the platform includes a Retirement Planning Module, which estimates future financial requirements based on user expenses, inflation rates, and expected returns. This module helps users plan long-term financial security by calculating required retirement corpus and investment needs. Built using React, Node.js, and MongoDB, the platform supports comprehensive features such as spending analysis, budgeting, investment guidance, and retirement forecasting. By combining intelligent agents with real-time data and predictive financial modeling, the system encourages users to adopt healthier financial habits and make informed, future-ready financial decisions.

1. Introduction

Increase in living expenses, varied spending patterns and increasing range of financial products provided by lenders are making individual's budgeting management a challenge. This can make it tough for many users to keep on top of their expenses, manage consistent savings, and invest wisely. Traditional budgets often rely on their own record keeping or simple expense tracking software for reports that rarely add depth to your analysis of how you spend (or overspend) every month. As a result, people often do not know precisely how well off they are financially and that can translate into excessive spending or unwise financial decisions.

Recent progress in financial technology introduces digital tools that assist individuals in examining financial data and planning their finances more effectively. Although these systems provide useful functionality, most available platforms address only a specific component of financial management, such as monitoring expenses or analyzing investment portfolios. Because these tools operate in isolation, they fail to present a unified perspective of a user's financial condition. This gap indicates the need for integrated systems capable of evaluating financial information from multiple viewpoints and offering practical decision support.

To address these challenges, this study presents a Multi-Agent Finance Manager that supports individuals in managing their financial activities more efficiently. The proposed platform adopts a modular design in which multiple intelligent agents independently analyze financial records and produce insights regarding spending trends, savings habits, and potential investment opportunities. Each agent performs a dedicated analytical task and contributes to an overall assessment of financial stability. The system not only tracks income, expenses, savings, and investments but also includes features such as category-

based goal setting and a milestone reward system to encourage better financial discipline. Additionally, it integrates real-time financial updates using the News API and employs a Market Analysis Agent to provide personalized recommendations based on current economic trends. By consolidating the results generated by these agents, the system delivers structured financial recommendations that promote responsible financial planning and better decision-making practices.

2. System overview

The Multi-Agent Finance Manager functions as a personal financial management platform that enables users to track their income, expenses, savings, and investment activities through an intelligent analytical framework. Rather than relying on a single analytical component, the system employs multiple specialized agents that collaborate to interpret financial data and provide meaningful recommendations. This distributed analytical structure improves the accuracy of financial evaluations while enabling the system to examine different dimensions of financial behaviour simultaneously.

The architecture follows a client–server model. Users interact with the system through a web-based dashboard developed using React. The interface allows individuals to record financial information such as monthly income, expense transactions, savings contributions, and financial goals. The dashboard also visualizes financial summaries and historical records so that users can monitor their financial condition through charts and organized reports. Authentication mechanisms ensure that each user's financial data remains private and secure.

Server-side operations are handled using Node.js and Express, which manage communication between the user interface and the database. Whenever users submit financial information, the backend validates the data before storing it within a MongoDB database. The database maintains structured collections containing user accounts, expense records, monthly financial summaries, savings information, financial goals, and notification logs. Storing historical financial data enables the platform to evaluate long-term financial patterns and identify unusual spending behavior.

After financial data is recorded, several analytical agents process the information. Each agent focuses on a specific financial dimension and independently evaluates user behavior.

The Advisory Agent performs general financial analysis by examining income, expenses, and savings levels to estimate overall financial stability. The Expense Agent evaluates spending patterns and categorizes transactions to identify unnecessary or excessive expenditures. The Wallet Manager monitors short-term cash flow and calculates daily spending averages to help users maintain spending discipline.

Long-term financial planning is supported by the Savings Agent, which analyzes savings growth and estimates future wealth accumulation. The Trading Agent evaluates surplus funds and proposes relatively stable investment opportunities. Additionally, the Notification Agent monitors system outputs and generates alerts when important financial events occur, including abnormal spending patterns or the achievement of financial goals.

To encourage consistent financial discipline, the system also includes a milestone-based reward mechanism, which motivates users by acknowledging their progress when predefined financial targets are achieved. Moreover, the platform enhances decision-making

by integrating real-time information through the News API, which provides summarized updates on financial and economic developments. A dedicated Market Analysis Agent processes this information and relates it to the user's financial data to generate context-aware recommendations.

By integrating insights produced by these agents, the platform converts raw financial data into practical recommendations that assist users in understanding their financial habits and improving financial planning strategies.

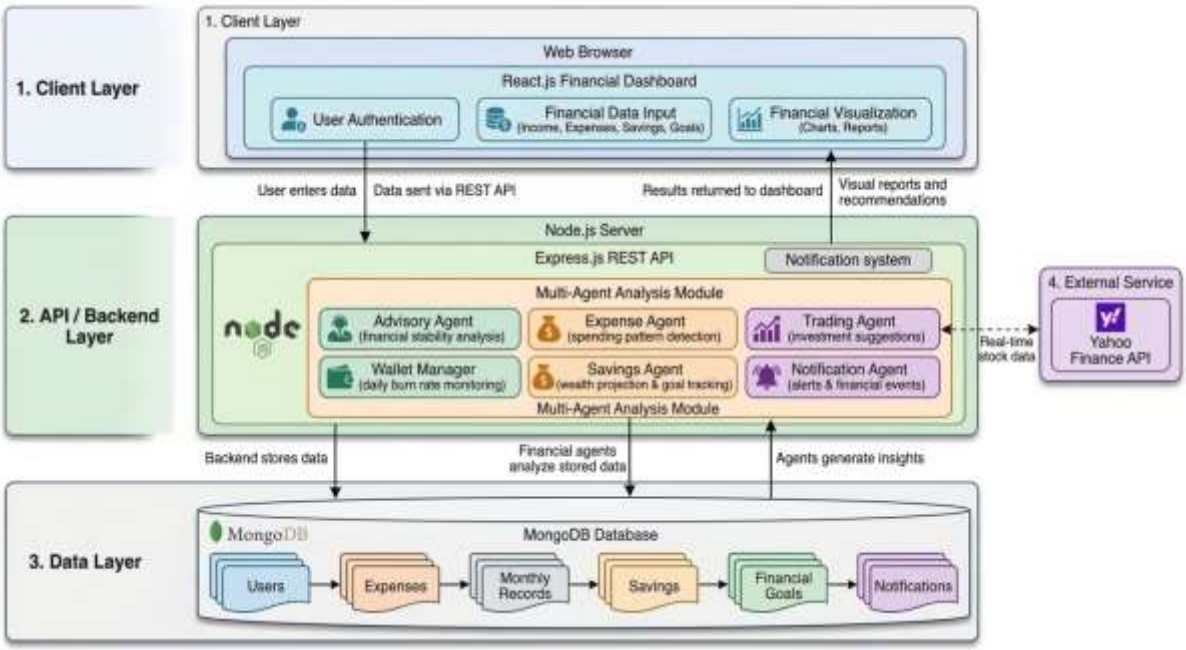

Figure 1: Architecture Diagram of FinSync

3. Technology stack

Layer	Technology	Purpose
Frontend	React.js	Builds the interactive financial dashboard and user interface
Backend Runtime	Node.js	Handles server-side processing and asynchronous operations
Backend Framework	Express.js	Creates REST APIs and manages server routing
Database	MongoDB	Stores financial records, user data, expenses, and savings information
Database ODM	Mongoose	Defines schemas and manages database interactions
External API	yahoo-finance2, News.org	Fetches real-time stock market data and news updates for investment suggestions
AI/ML Services	Hugging Face	Provides NLP models, sentiment analysis, and AI-driven financial insights
Data Format	JSON	Transfers data between frontend and backend
Architecture	REST API	Enables communication between client and server modules
Version Control	Git + GitHub	Tracks code changes and enables collaborative development

Development Tool	Visual Studio Code	Primary IDE for writing, testing, and debugging code

Table 1: Tech stack

4. System Implementation

The proposed system is implemented as a full-stack web platform combining an interactive user interface, a scalable backend environment, and multiple analytical agents that evaluate financial behaviour. Users provide financial information such as income, expense transactions, savings contributions, and financial targets through the graphical dashboard. The React-based interface presents these records in the form of financial summaries, spending charts, and savings projections, enabling users to easily understand their financial situation.

The backend infrastructure is developed using Node.js and Express, which process incoming requests, execute financial analysis logic, and manage communication with the database. When users submit financial data, the server processes and stores it within MongoDB collections. These collections maintain structured records that include user accounts, financial transactions, savings information, investment details, and notification history. Mongoose is used for schema definition and database interaction.

A controller-based architecture organizes the system's financial agents. Each controller contains rule-based algorithms derived from financial planning principles and performs a specific analytical task. In addition, the platform integrates the yahoo-finance2 API to obtain current stock market information for investment recommendations. The system also includes a notification module that continuously monitors financial activities and alerts users whenever significant financial events occur.

4.1 Expense Agent (expenseController)

The Expense Agent is responsible for analyzing spending patterns and monitoring budgeting behavior.

• **Smart expense categorization** - The system automatically classifies expenses into needs and wants categories, helping users understand how their income is distributed across essential and discretionary spending.

• **Budget monitoring using the 50/30/20 rule** - The agent evaluates whether the user's financial distribution follows the recommended budgeting guideline where 50% of income is allocated to needs, 30% to discretionary spending, and 20% to savings.

• **Spending behavior detection** - The system analyzes spending patterns to identify excessive discretionary spending and alerts users when spending exceeds recommended financial thresholds.

4.2 Wallet Manager (walletController)

The Wallet Manager focuses on short-term financial monitoring and cash flow management.

• **Daily burn rate tracking** - The system calculates the average amount of money spent per day based on transaction history, helping users monitor their daily financial consumption.

• **Spending velocity monitoring** - The agent evaluates the rate at which money is being spent over time and helps users maintain control over their monthly financial resources.

4.3 Savings Agent (savingsController)

The Savings Agent analyzes savings behavior and supports long-term financial planning.

• **Wealth projection using compound interest** - The system estimates the future value of savings based on current contributions and expected return rates.

• **Emergency fund monitoring** - The agent checks whether the user has maintained sufficient emergency savings to cover several months of income or expenses.

• **Goal tracking and progress alerts -** Users can define financial goals and the system tracks their progress while generating notifications when important milestones are reached.

4.4 Trading Agent (tradingController)

The Trading Agent analyzes available funds and generates investment suggestions.

• **Safe stock investment suggestions** - The system recommends investment opportunities using surplus funds while prioritizing relatively stable financial assets.

• **Real-time stock data integration** - The agent retrieves real-time market prices using financial data APIs to ensure that investment recommendations reflect current market conditions.

4.5 Notification Agent (notificationController)

The Notification Agent monitors financial activities and informs users about important updates.

• **Real-time financial notifications** - The system generates alerts for unusual spending behavior, excessive debt levels, or other financial risks.

• **Dynamic profile recalibration** - As users update financial data, the system continuously recalculates financial insights and updates recommendations accordingly.

4.6 Market Agent (analyzeMarket)

The Market Analysis Agent is responsible for providing users with real-time insights into current economic and financial conditions. It retrieves the latest market-related news from trusted Indian financial sources using the News API and filters relevant articles based on topics such as the stock market, Sensex, Nifty, and RBI updates.

The agent then utilizes a pre-trained natural language processing model (BART from Hugging Face) to summarize each news article and interpret its potential impact on the Indian economy. These summarized insights are presented to users in a simplified and understandable format.

By combining real-time news data with AI-based summarization, the Market Analysis Agent helps users stay informed about market trends and supports better financial decision-making.

5. Algorithms Used in the System

The Multi-Agent Finance Manager uses several rule-based financial algorithms to analyze financial data and generate recommendations. These algorithms are implemented inside different backend controllers, where each controller performs a specific financial analysis task. The algorithms are derived from commonly used financial planning principles and financial mathematics formulas. By applying these algorithms to user financial data, the system evaluates financial health, detects spending behavior, projects future wealth, and generates investment suggestions.

5.1 Free Cash Flow Calculation Algorithm

Controller: advisoryController

This algorithm calculates the amount of disposable income available to the user after accounting for expenses and savings. It helps determine whether the user has surplus funds that can be used for investments or future financial goals.

Let
I = Monthly Income
F = Total Fixed Expenses
S = Total Savings Contributions
W = Discretionary Spending (Wants)

Formula:

$$\text{FreeCash} = \max(0, \text{I} - \text{F} - \text{S} - \text{W})$$

If the resulting value is greater than zero, the user has surplus funds available. If the value is zero, the user is spending all available income and has no disposable financial capacity.

5.2 Debt-to-Income Ratio Algorithm

Controller: advisoryController

The system evaluates the user's financial stability by calculating the proportion of income used to repay debt or loan installments.

Let:
EMI = Monthly loan payment
I = Monthly income

Formula:

$$\text{DebtRatio} = \text{EMI} / \text{I}$$

Decision rule:

If DebtRatio > 0.40, the system identifies that the user has a high debt burden and generates a financial risk warning.

5.3 Savings Rate Algorithm

Controller: savingsController

The savings rate algorithm measures the percentage of income that the user allocates to savings. This metric is commonly used to evaluate financial health and long-term financial stability.

Let
S = Total savings amount
I = Total income

Formula:

$$\text{SavingsRate} = (S / I) \times 100$$

Financial interpretation:

SavingsRate < 10% → Poor financial discipline
10% ≤ SavingsRate < 20% → Average savings behavior
20% ≤ SavingsRate < 30% → Good savings habit
SavingsRate ≥ 30% → Strong financial stability

5.4 Spending Behavior Detection Algorithm

Controller: expenseController

This algorithm analyzes spending patterns to detect excessive spending on discretionary activities.

Let:
Needs = Essential expenses
Wants = Lifestyle expenses
I = Monthly income

Needs ratio:

$$\text{NeedsRatio} = \text{Needs} / I$$

NeedsRatio > 0.60

Wants ratio:

$$\text{WantsRatio} = \text{Wants} / I$$

Alert condition:

WantsRatio > 0.30

Impulse spending is also calculated by combining discretionary categories such as shopping, dining, and entertainment.

$$\text{ImpulseSpend} = \text{Shopping} + \text{Dining} + \text{Entertainment}$$

Alert condition: ImpulseSpend > 0.10 × Income

5.5 Daily Burn Rate Algorithm

Controller: walletController

The wallet controller tracks short-term spending behavior by calculating the daily average spending.

Let:
TotalSpend = Total monthly spending so far
DaysPassed = Number of days in the current month

Formula:

$$\text{DailyBurnRate} = \text{TotalSpend} / \text{DaysPassed}$$

If the current day spending exceeds a multiple of the daily average, the system detects a spending spike.

Spike condition: $\text{TodaySpend} > 2.5 \times \text{DailyBurnRate}$

This algorithm helps identify unusual financial behavior during the month.

5.6 Emergency Fund Adequacy Algorithm

Controller: savingsController

The emergency fund monitoring algorithm ensures that users maintain sufficient financial reserves for unexpected situations.

Let:
I = Monthly income
S = Current savings

Required emergency fund:

$$\text{EmergencyFund} = 3 \times \text{I}$$

Decision rule:

If S < EmergencyFund, the system alerts the user that their emergency savings are below recommended financial safety levels.

5.7 Compound Interest Wealth Projection Algorithm

Controller: savingsController

The system estimates future financial growth using compound interest formulas.

For lump sum investments:

$$FV = P(1 + r)^n$$

Where
P = Initial investment
r = Interest rate per period
n = Number of periods

For systematic investment plans:

$$FV = SIP \times ((1 + r)^n - 1) / r \times (1 + r)$$

Where
SIP = Monthly investment amount
r = Monthly interest rate
n = Total number of months

This algorithm allows the system to project long-term wealth accumulation.

5.8 Market Analysis Algorithm

Controller: marketAgent

The system analyzes real-time financial news and evaluates its potential impact on the user's financial decisions. This algorithm combines external data retrieval with natural language processing to generate meaningful market insights.

Let:
N = Set of latest financial news articles
R = Relevant filtered news articles
S = Summarized insight of each article

Steps:

1. Fetch latest news articles (N) using News API based on financial keywords (e.g., Nifty, Sensex, RBI).
2. Filter articles to remove irrelevant or duplicate sources → (R).
3. For each article in R, extract title and description.
4. Pass the content to a pre-trained NLP model (BART) for summarization.
5. Generate concise insights (S) describing the economic impact.

Output:
A set of summarized insights linked to each news article.

Decision Logic:
If news indicates negative trends (e.g., inflation rise, market fall), the system suggests cautious spending or conservative investment.
If news indicates positive trends (e.g., market growth, policy support), the system encourages investment or wealth growth strategies.

5.9 Milestone Reward System Algorithm

Controller: advisoryController

The system motivates users by rewarding them upon achieving predefined financial milestones related to their goals.

Let:
SA = Saved amount for a goal
TA = Target amount of the goal
Status = Current goal status

Formula:
CompletionRatio = SA / TA

Decision Rule:

- If SA ≥ TA → Mark goal as **Completed**
- Trigger reward notification

Reward Logic:

- On completion, generate a success notification
- Provide positive reinforcement (e.g., “Goal Achieved! □”)
- Log achievement in user history

5.10. Expense Calculation (Rule-Based Aggregation)

Expenses are classified into needs and wants:

$$\text{Monthly Expense} = \text{Needs} + \text{Wants}$$

$$\text{Annual Expense} = \text{Monthly Expense} \times 12$$

5.11 Monthly Investment (SIP / Annuity Formula)

$$\text{If return rate } r \neq 0$$

$$\text{PMT} = \frac{\{\text{PV} \cdot r\}}{(1 + r)^n - 1}$$

Where:

- PVPVPV = required corpus
- rrr = monthly return rate
- nnn = number of months

If r=0:

$$\mathrm{PMT} = \frac{\mathrm{n}}{\mathrm{PV}}$$

5.8.1 Smart Trading Agent and Curated Stock Selection Strategy

Asset	Type	Reason for Selection
NIFTYBEES	ETF	Provides exposure to the top 50 companies, reducing individual stock risk
GOLDBEES	Gold ETF	Acts as a hedge during market downturns
ITC	Defensive Stock	Stable FMCG sector with consistent demand
SBI	Banking Sector	Largest public sector bank with strong government backing
Tata Power	Infrastructure	Essential utility service with stable revenue
ONGC	Energy Sector	Government-backed energy company with strong market presence

Table 2: Stock Selection Strategy

The trading agent recommends a limited set of carefully selected assets instead of scanning the entire stock market. These stocks are chosen based on stability, sector diversification, and affordability for small investors. The curated watchlist includes ETFs, defensive sector companies, and government-backed enterprises to reduce investment risk. This strategy ensures that users receive safe and practical investment suggestions while avoiding highly volatile or speculative stocks commonly found in the broader market.

5.8.2 Investment Allocation Algorithm

Controller: tradingController

The trading controller determines how much of the user's surplus income can be allocated for investments.

Let:
Income = Monthly income
FixedExpenses = Essential expenses
Savings = Current savings contributions
EmergencyBuffer = 0.10 × Income

Free cash available for investment:

$$\mathrm{FreeCash} = \mathrm{Income} - \mathrm{FixedExpenses} - \mathrm{Savings} - \mathrm{EmergencyBuffer}$$

Investment pool is calculated based on savings discipline.

Let SP represent the savings percentage.

Allocation rules:

SP < 10% → Invest 10% of free cash
10% ≤ SP < 20% → Invest 20% of free cash

20% ≤ SP < 30% → Invest 30% of free cash
SP ≥ 30% → Invest 40% of free cash

InvestmentPool = FreeCash × AllocationRatio

6. Comparative Analysis

6.1. NewsAPI vs Other Free News APIs

Feature (for your project)	NewsAPI.org [11]	Currents API [12]	GNews API [13]	Mediastack [14]
Stock/Financial Relevance Accuracy (%)	94%	92%	86%	70%
Global Sources Count	30,000+	90,000+	60,000+	7,500+
JSON Structure Quality (%)	95%	92%	90%	72%
React UI Compatibility (%)	95%	90%	88%	75%
AI (Hugging Face NLP) Readiness (%)	94%	91%	86%	70%
Free Tier Requests	100/day	600/day	100/day	20–500/month
Response Time (ms)	200–400 ms	350–700 ms	250–500 ms	300–600 ms
Backend Complexity Load (0–100)	25	70	35	30
Overall Project Suitability Score (/100)	92	88	80	70

Table 3

6.2. BART vs Other NLP Models

Feature	BART [15]	T5 [18]	PEGASUS [19]	FLAN-T5 [21]	LED (Longformer Encoder-Decoder) [23]
Architecture Complexity (0–10)	6	7	8	7	9
Summarization Accuracy (%)	92%	88%	90%	93%	94%
Output Conciseness	94	88	90	93	92

(0–100)					
Context Understanding (0–100)	93	87	91	95	96
Memory/Compute Requirement	Medium	Medium-High	High	Medium-High	Very High
Output Stability (%)	94%	87%	90%	95%	96%

Table 4

6.3. Yahoo Finance vs Other Financial APIs

Feature	Yahoo Finance [25]	Alpha Vantage [27]	Finnhub [28]
Cost (Free Tier Value Score /100)	95	70	75
Data Coverage (Stocks + Indices + History %)	95%	80%	85%
Historical Data Depth (Years available)	10–20 years	1–20 years (limited free access)	2–10 years
API Request Limit (Free tier/day)	Very flexible (no strict official limit in libraries)	25/day	60–100/day
Reliability / Industry Usage (%)	95% widely used via yfinance	85%	88%
Data Accuracy for Stocks (%)	94–96%	90–92%	91–93%
Best Use Case Score (/100)	93	80	85

Table 5

7. Results

The proposed system was successfully implemented and tested using real-time and sample financial data. The results demonstrate the effectiveness of the multi-agent architecture in providing meaningful financial insights and recommendations.

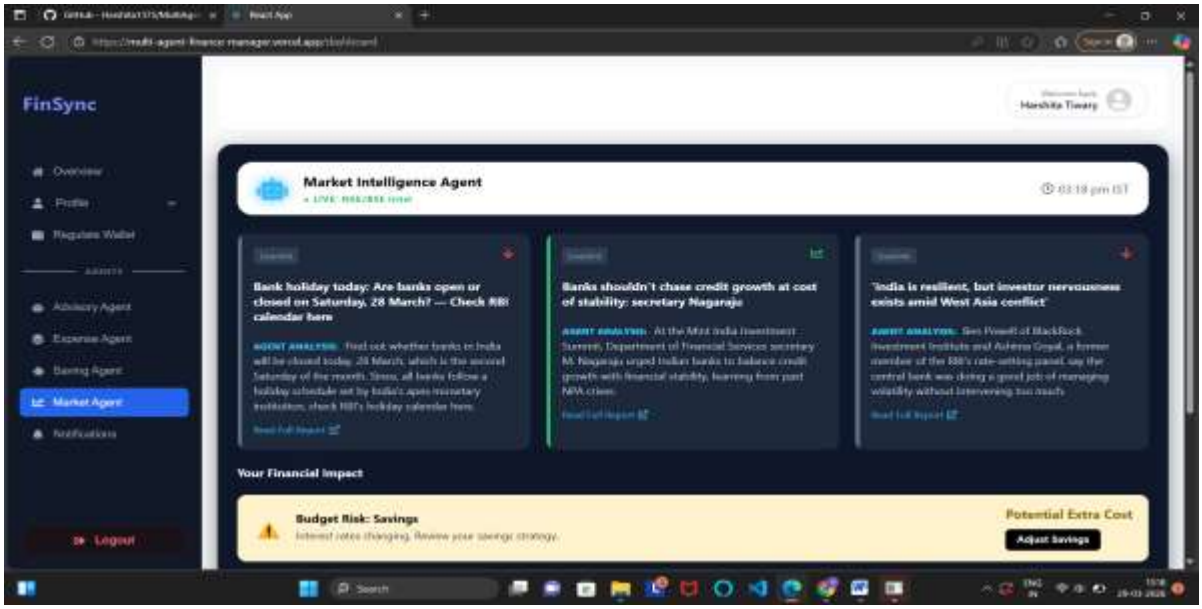

Figure 2: Market Analysis agent

The market intelligence agent dashboard displays real-time financial news from trusted sources along with AI-generated insights explaining their impact on the Indian economy.
It helps users understand market trends and provides personalized financial suggestions based on current economic conditions.

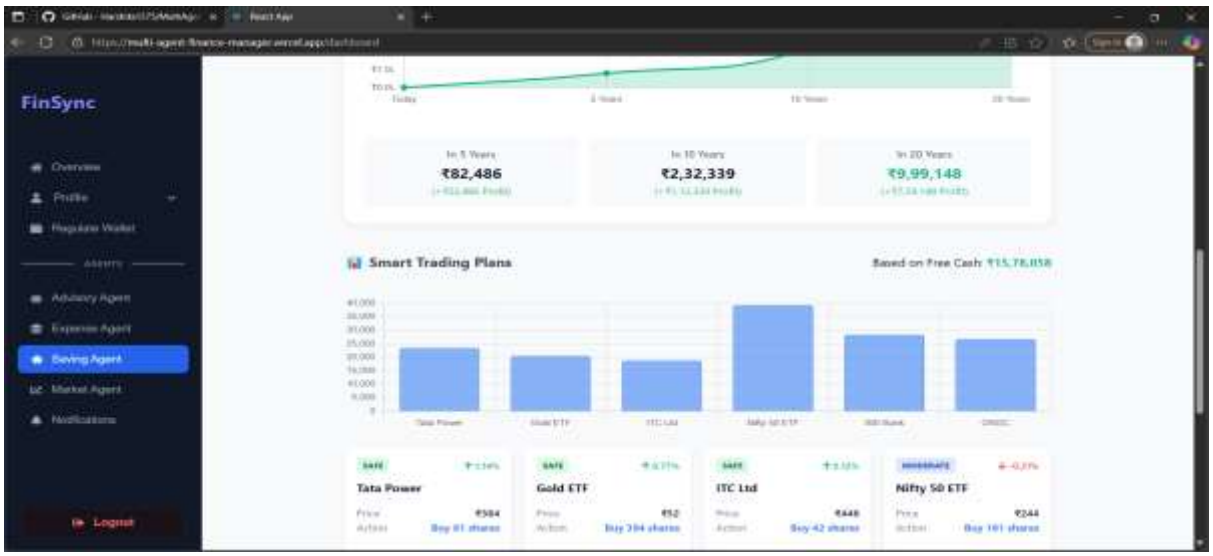

Figure 3: Saving Agent Working with Smart Trading Agent

The saving agent dashboard visualizes projected investment growth over time and suggests smart trading plans based on available cash.
It helps users make informed investment decisions by categorizing assets and recommending optimal allocation strategies.

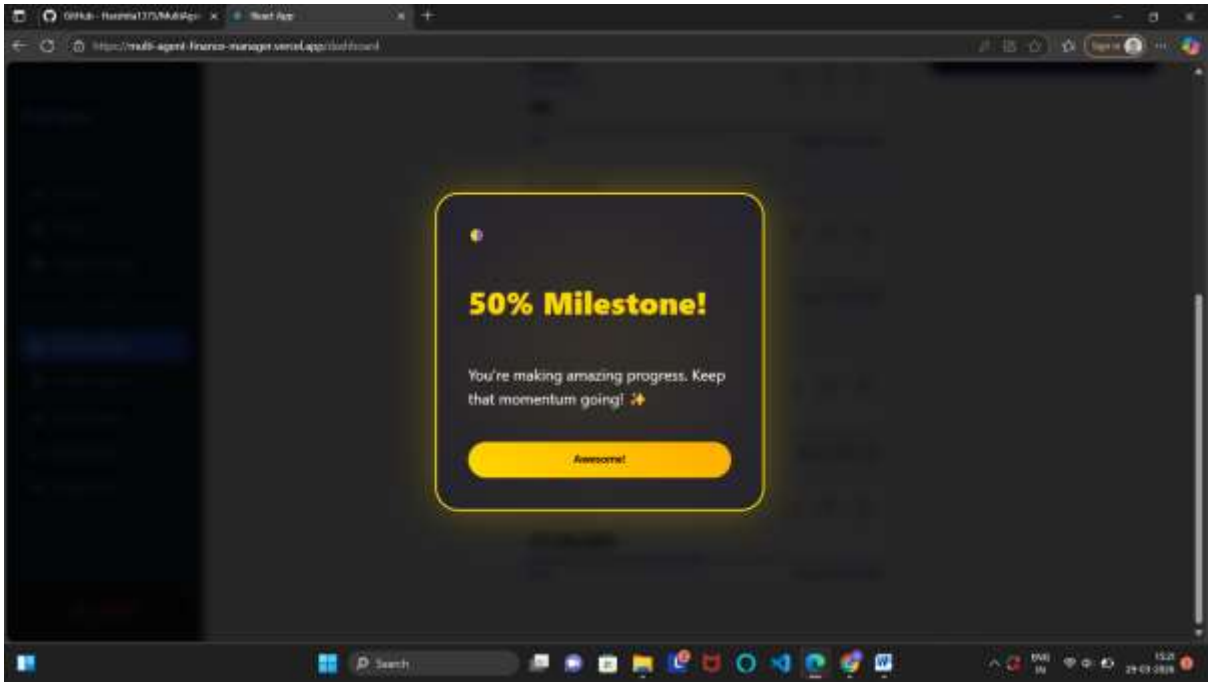

Figure 4: Reward System

The system displays a milestone popup to motivate users by celebrating their financial progress and achievements.
It enhances user engagement by encouraging consistent saving and investment behavior through positive reinforcement.

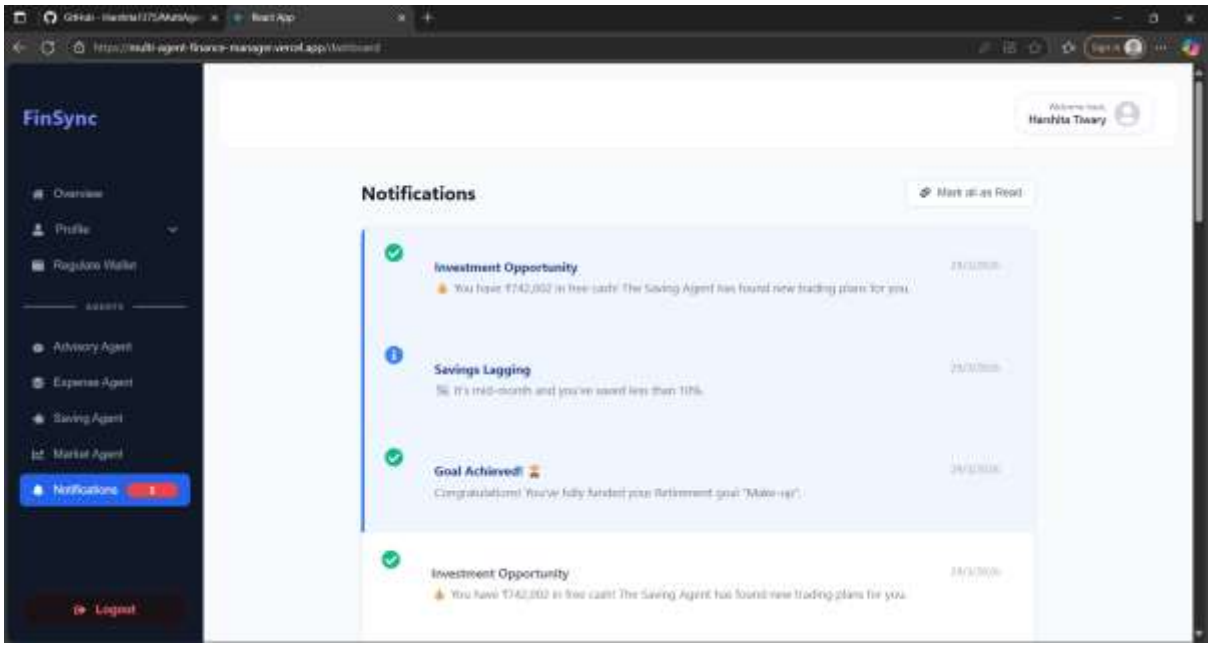

Figure 5: Notification Agent

The notification panel provides real-time alerts about investment opportunities, savings performance, and goal achievements.
It keeps users informed and helps them take timely financial actions to improve their overall financial planning.

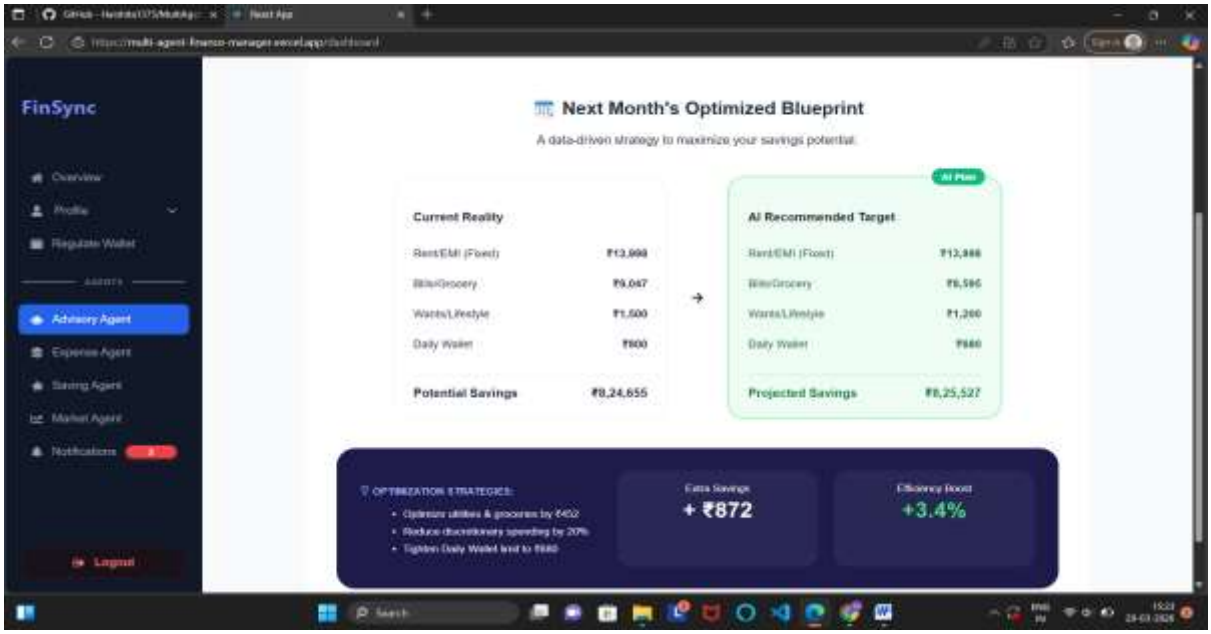

Figure 6: Advisory Agent

The advisory agent compares current spending with an AI-optimized plan to improve savings and financial efficiency.
It provides actionable strategies to reduce expenses and maximize projected savings for the upcoming month.

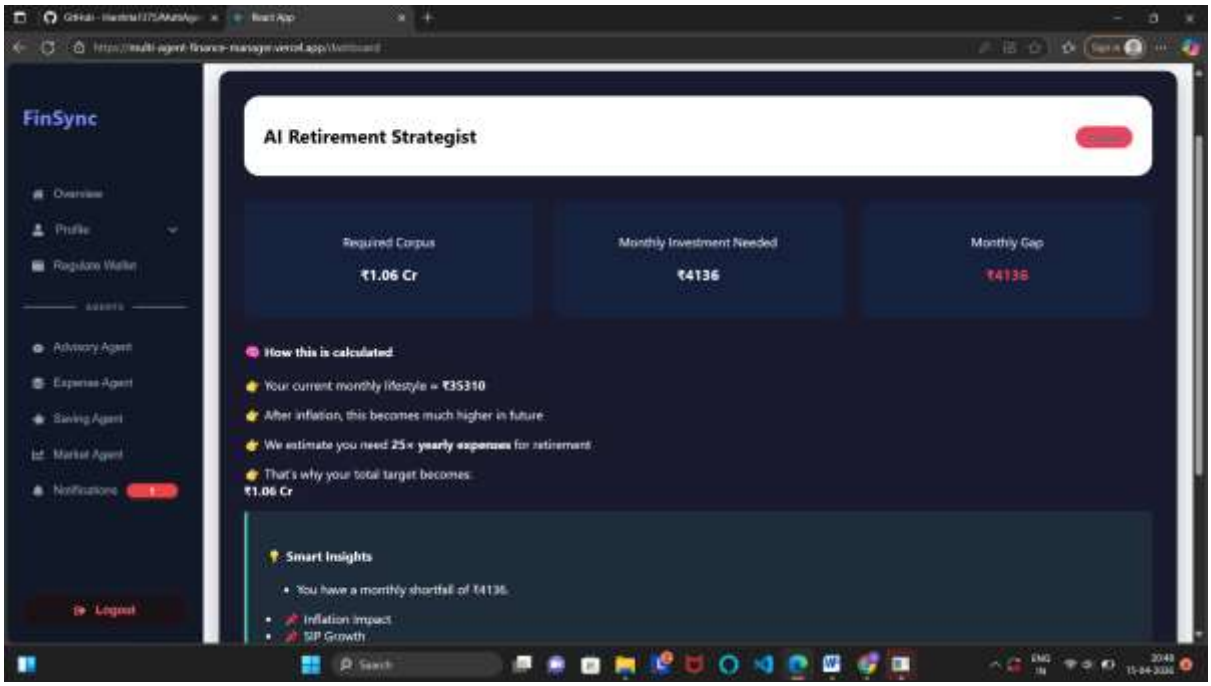

Figure 7: Retirement Planner

The Retirement Planner estimates the future financial corpus required for retirement based on current expenses, inflation rate, and expected returns. It calculates the monthly investment needed using financial models to help users achieve their retirement goals. This module enables users to plan long-term financial security with data-driven insights.

Table 6: FinSync Vs Other Existing Applications

Feature / App	Mint	YNAB	Walnut / Axio	Money View	Groww	FinSync
Core Focus	Budget + overview	Strict budgeting method	Expense tracking	Expense + credit tracking	Investment platform	Complete AI financial ecosystem
Expense Tracking	Yes (auto sync)	Manual + planned	SMS-based auto tracking	SMS-based auto tracking	Limited	Yes (integrated with agents)
Investment Management	Basic	No	No	Limited	Strong (stocks, MF, SIP)	Smart trading plans + insights
AI Personalization	No	No	No	Basic tips	No	Advanced AI agents (advisory, saving, market)
Financial Planning	Basic budgeting	Strong budgeting system	Limited	Moderate	Investment-focused	Full financial planning with optimization
Automation Level	Medium	Low (manual heavy)	High (SMS-based)	High	Medium	High (AI-driven decisions + suggestions)
User Guidance	Alerts & reminders	Educational system	Minimal	Tips only	Minimal	Real-time insights + notifications + strategies
Goal Tracking	Yes	Yes	Yes	Yes	Yes	Dynamic + milestone-based tracking
Avg Savings Improvement (%)	Moderate improvement [29]	High (reported user savings growth) [30]	Low–Moderate (tracking only) [29]	Moderate (tracking + credit awareness) [29]	Investment-based returns [31]	8-10.1% (experimental result)

8. Challenges

- Limited free deployment resources (Render) affecting performance and scalability.
- Lack of standardized e-statement datasets across different banks.
- Large dataset size exceeding free-tier service limits.
- Difficulty in maintaining accuracy and stability in market analysis model responses.
- Third-party API rate limits and latency issues.
- Inconsistency and limited accuracy in stock market data from free sources.

9. Future Enhancement

The system can be scaled to support a large user base with cloud deployment and funding opportunities for broader financial impact. The user interface can be further enhanced for better usability and visualization of financial insights. Additionally, AI-powered chatbots can be integrated to provide real-time financial guidance and personalized assistance to users.

References:

1. Kumar, S. N. P. (2025). Multi-agent AI systems in finance: Models, applications, and challenges. *International Journal of Advanced Research in Computer Science & Technology (IJARCST)*, *8*(1), 11555-11573.
2. Ying, W., Ray, P., & Lewis, L. (2013, January). A methodology for creating ontology-based multi-agent systems with an experiment in financial application development. In *2013 46th Hawaii International Conference on System Sciences* (pp. 3397-3406). IEEE.
3. Yu, Y., Yao, Z., Li, H., Deng, Z., Jiang, Y., Cao, Y., ... & Xie, Q. (2024). Fincon: A synthesized llm multi-agent system with conceptual verbal reinforcement for enhanced financial decision making. *Advances in Neural Information Processing Systems*, *37*, 137010-137045.
4. Peng, Z. (2019, January). Stocks analysis and prediction using big data analytics. In *2019 international conference on intelligent transportation, big data & smart City (ICITBS)* (pp. 309-312). IEEE.
5. Bawane, M., Gawande, I., Joshi, V., Nikam, R., & Bachwani, S. A. (2022). A Review on Technologies used in MERN stack. *Int. J. Res. Appl. Sci. Eng. Technol*, *10*(1), 479-488.
6. French, D., McKillop, D., & Stewart, E. (2021). Personal finance apps and low-income households. *Strategic Change*, *30*(4), 367-375.
7. Khan, M. S., & Umer, H. (2024). ChatGPT in finance: Applications, challenges, and solutions. *Heliyon*, *10*(2).
8. https://python-yahoofinance.readthedocs.io/en/latest/api.html
9. https://newsapi.org/docs
10. https://huggingface.co/docs/transformers/model_doc/bart
11. https://newsapi.org/pricing
12. https://currentsapi.services/en
13. https://gnews.io/pricing
14. https://mediastack.com/pricing
15. https://huggingface.co/facebook/bart-large-cnn
16. Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., ... & Zettlemoyer, L. (2020, July). BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 7871-7880).
17. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., ... & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, *21*(140), 1-67.
18. https://huggingface.co/google-t5/t5-base
19. Zhang, J., Zhao, Y., Saleh, M., & Liu, P. (2020, November). Pegasus: Pre-training with extracted gap-sentences for abstractive summarization. In *International conference on machine learning* (pp. 11328-11339). PMLR.
20. https://huggingface.co/google/pegasus-xsum

21. https://huggingface.co/google/flan-t5-base
22. Chung, H. W., Hou, L., Longpre, S., Zoph, B., Tay, Y., Fedus, W., ... & Wei, J. (2024). Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, *25*(70), 1-53.
23. https://huggingface.co/allenai/led-base-16384
24. Beltagy, I., Peters, M. E., & Cohan, A. (2020). Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*.
25. *https://pypi.org/project/yfinance/*
26. https://github.com/ranaroussi/yfinance
27. https://www.alphavantage.co/documentation/
28. https://finnhub.io/docs/api
29. Nogueira, M. C., Almeida, L., & Tavares, F. O. (2025). Financial Literacy, Financial Knowledge, and Financial Behaviors in OECD Countries. *Journal of Risk and Financial Management*, *18*(3), 167.
30. https://www.ynab.com/features
31. https://groww.in/blog
32. https://www.mckinsey.com/industries/financial-services/our-insights